

34

LlamaIndex

A framework that makes building RAG easy

All the RAG plumbing, handled for you

[AI/ML Engineer Learning Series · Deep Dive Edition](#)

What it is	A framework for building RAG apps fast
Handles	Loading, chunking, embedding, indexing, retrieval
The win	Full RAG in a few lines instead of dozens
Focus	Connecting your data to LLMs
Pairs with	FAISS, vector DBs, any LLM

What's Inside

Here is everything covered in this guide, in order:

#	Section	What you'll learn
1	What is LlamaIndex?	The RAG framework
2	Why Use a Framework?	Dozens of lines vs a few
3	The Core Concepts	Documents, nodes, index, query engine
4	The Simplest RAG App	Five lines that work
5	Loading Different Data	PDFs, web, folders
6	Customising the Pipeline	Chunking, embeddings, top-k
7	LlamaIndex vs LangChain	Which framework when
8	Common Mistakes	What to avoid
★	Cheatsheet	Quick reference

1. What is LlamaIndex?

In the last topic you built RAG from scratch — chunking, embedding, indexing, searching, prompting. That's a lot of moving parts to wire together by hand. LlamaIndex is a framework that does all of that plumbing for you. It's built specifically to connect your data to LLMs, which is exactly what RAG needs.

With LlamaIndex, the whole RAG pipeline you just learned — load documents, split them, embed them, store them, retrieve, and answer — becomes just a few lines of code. It handles the boring, fiddly parts so you can focus on your actual app.

■ *Now that you understand RAG from the ground up (last topic), a framework like LlamaIndex makes sense. You know what it's doing under the hood, so you're not just copying magic code — you understand each step it automates.*

2. Why Use a Framework?

Building RAG by hand means writing and connecting many pieces: a PDF loader, a chunker, an embedder, an index, a search function, a prompt builder, and the LLM call. Each is a chance for bugs. A framework bundles all of this into clean, tested, ready-made tools.

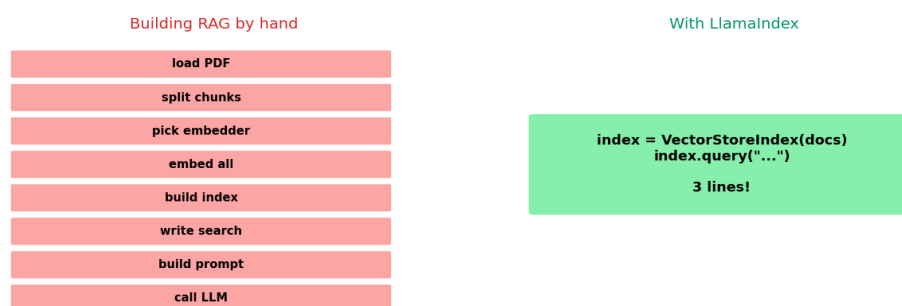


Fig 1 — Building RAG by hand is many steps. LlamaIndex collapses them into a few lines.

The benefit isn't just less code — it's fewer mistakes, faster building, and easy access to advanced features (better retrieval, many data sources, different vector stores) that would take ages to build yourself. You write less and get more.

3. The Core Concepts

LlamaIndex has a few key building blocks. Knowing these names makes everything else click into place.



Fig 2 — The core LlamaIndex concepts, from raw data to a full question-answering engine.

Concept	What it is
Document	A piece of your raw data (a PDF, a web page, text)
Node	A chunk — a small piece of a document
Index	The searchable store of your embedded nodes
Retriever	The part that finds the relevant nodes for a question
Query Engine	The full thing: retrieve + ask the LLM + return answer

Notice how these map exactly onto the RAG steps you learned: Documents are your data, Nodes are the chunks, the Index is your vector store, the Retriever does retrieval, and the Query Engine ties it all together into a question-answering system. Same concepts, friendlier names.

4. The Simplest RAG App

Here's a complete, working RAG app over a folder of documents — in about five lines. Compare this to all the manual code from the last topic.

```
# pip install llama-index
from llama_index.core import VectorStoreIndex, SimpleDirectoryReader

# 1. Load all documents from a folder
documents = SimpleDirectoryReader('my_docs').load_data()

# 2. Build the index (chunks + embeds + stores - all automatic)
index = VectorStoreIndex.from_documents(documents)

# 3. Make a query engine and ask questions
query_engine = index.as_query_engine()
response = query_engine.query('What does the document say about leave?')
print(response)
```

That's a full RAG system. Behind those lines, LlamaIndex loaded the files, split them into nodes, embedded them, built a searchable index, and set up retrieval plus the LLM call. Everything from the last topic, automated.

■ *This is the power of a framework: the five lines above do what took a full page of code by hand. And because you learned the manual version first, you know exactly what each line is doing.*

5. Loading Different Data

A big strength of LlamaIndex is loading data from almost anywhere. It has ready-made readers for many sources, so getting your data in is easy.

Source	How
A folder of files	SimpleDirectoryReader('folder')
PDFs	Handled automatically by the directory reader
Web pages	Web reader plugins
Databases / APIs	Many connectors available (LlamaHub)
Notion, Slack, Google Drive	Ready-made connectors

LlamaHub is a collection of hundreds of these data connectors. Whatever your data lives in, there's usually a reader for it — so you spend your time building, not writing loaders. For your PDF chatbot, the directory reader handles PDFs out of the box.

6. Customising the Pipeline

The simple version uses sensible defaults, but you can control every part when you need to — chunk size, which embedding model, how many chunks to retrieve, and which LLM.

```
from llama_index.core import VectorStoreIndex, SimpleDirectoryReader, Settings
from llama_index.embeddings.huggingface import HuggingFaceEmbedding

# Choose your embedding model (e.g. the one you know)
Settings.embed_model = HuggingFaceEmbedding(
    model_name='all-MiniLM-L6-v2')

# Control chunk size
Settings.chunk_size = 512
Settings.chunk_overlap = 50

docs = SimpleDirectoryReader('my_docs').load_data()
index = VectorStoreIndex.from_documents(docs)

# Retrieve more chunks per question
query_engine = index.as_query_engine(similarity_top_k=5)
print(query_engine.query('your question'))
```

■ Notice `similarity_top_k=5` — that's the top-k retrieval setting from the RAG topic. And you can plug in the same `all-MiniLM-L6-v2` embedder you used directly. The framework wraps the concepts you already know.

7. LlamaIndex vs LangChain

You'll hear about two big RAG/LLM frameworks: LlamaIndex and LangChain (next topic). They overlap a lot, but have different focuses. You don't have to pick just one — many projects use both.

	LlamaIndex	LangChain
Main focus	Data + RAG (search your docs)	Chains + agents (workflows)
Best at	Connecting data to LLMs	Building multi-step LLM apps
RAG	Its core strength	Also supported
Feel	Data-centric, focused	Broad, flexible toolkit
Use when	Your app is mainly about your docs	Your app has complex logic/tools

Simple way to choose: if your app is mostly 'answer questions about my documents', LlamaIndex shines. If it's a more complex workflow with multiple steps, tools, and decisions, LangChain (next) is built for that. Both are excellent, and they can work together — LlamaIndex for retrieval, LangChain for the wider app logic.

8. Common Mistakes

Mistake	Why it's a problem	Fix
Using it without knowing RAG	Magic code you can't fix	Learn RAG first (you did!)
Default chunk size always	May not fit your docs	Tune <code>chunk_size</code> / <code>overlap</code>
Retrieving too few chunks	Misses answers	Raise <code>similarity_top_k</code>
Wrong embedding model	Weak retrieval	Pick a good embedder explicitly
Re-indexing every run	Slow and wasteful	Save and reload the index
Ignoring LangChain fit	Wrong tool for complex apps	Use LangChain for workflows

Quick Reference Cheatsheet

Concept / Task	Meaning or Code
LlamaIndex	framework for building RAG fast
Document	a piece of raw data
Node	a chunk of a document
Index	searchable store of embedded nodes
Query engine	retrieve + LLM = full Q&A
Install	<code>pip install llama-index</code>

Load folder	<code>SimpleDirectoryReader('docs').load_data()</code>
Build index	<code>VectorStoreIndex.from_documents(docs)</code>
Make engine	<code>index.as_query_engine()</code>
Ask	<code>query_engine.query('question')</code>
Set chunk size	<code>Settings.chunk_size = 512</code>
More results	<code>as_query_engine(similarity_top_k=5)</code>
Best for	data-focused RAG apps

The one idea to remember: LlamaIndex automates the whole RAG pipeline you learned by hand — loading, chunking, embedding, indexing, retrieving, and answering — in just a few lines. It's the fast way to connect your data to an LLM, and it shines for document question-answering.

Next Topic: LangChain — the broader framework for building multi-step LLM applications: chaining calls together, using tools, and creating the workflows that lead into AI agents.